PROJECT REPORT

ONLINE SHOPPING CART AND ORDER PROCESSING SYSTEM

Java Course-End Project | BTL-5 – Evaluate

Submitted by	Roll Number
R. Sayi Parthu	160125737119
R. Charan Tej Reddy	160125737122

Under the Guidance of

K. Swati

Assistant Professor, Department of Information Technology

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
(CBIT) DEPARTMENT OF INFORMATION TECHNOLOGY (IT)
KOKAPET (V), GANDIPET, RANGA REDDY, HYDERABAD – 500075
www.cbit.ac.in

GitHub Repository: https://github.com/charanreddy-73/Java-CourseEndProject

1. PROJECT DESCRIPTION

1.1 Project Overview

The Online Shopping Cart and Order Processing System is a Java-based console application developed to manage product selection, customer shopping carts, price calculations, product filtering, and order processing. It demonstrates the practical use of the Java Collections Framework, Lambda Expressions, Functional Interfaces, the Stream API, Serialization, Deserialization, and Multithreading.

The application processes three customers. Each customer can enter products from a predefined catalogue, view the contents of their cart, calculate the total and average price, filter items using predefined conditions, and save and restore cart information through object serialization.

1.2 Problem Statement

Design an online shopping cart application to manage products, customer carts, and orders. Use an ArrayList to maintain cart items and a Map to maintain product details and prices. Implement Lambda Expressions and Functional Interfaces to filter products based on price and category. Use Serialization and Deserialization to save and restore customer cart information. Compare the performance and suitability of List and Map collections for different shopping operations.

1.3 Objectives

- Develop a Java-based shopping cart and order processing application.
- Use ArrayList to store the products selected by each customer.
- Use a Map to associate product names with their prices.
- Calculate the total and average price of the items in a cart.
- Apply Lambda Expressions and a custom Functional Interface to filter products.
- Save and restore customer cart objects using Serialization and Deserialization.
- Demonstrate concurrent processing using multiple customer threads.
- Evaluate the suitability of List and Map collections for shopping operations.

1.4 Technologies and Packages Used

Technology / Package / Concept	Purpose in the Project
java.util	Provides Scanner, List, ArrayList, Map, Collections, and other utility classes.
java.io	Provides Serializable, FileOutputStream, FileInputStream, ObjectOutputStream, and ObjectInputStream.
java.util.stream	Supports stream-based processing of cart items.
ArrayList	Maintains the ordered list of products selected by a customer.
Map	Associates each product name with its price for lookup and calculation.
Lambda Expressions	Provide compact implementations of price-based and category-based filtering.
Functional Interface	Defines the ProductFilter operation implemented by lambda expressions.
Serialization / Deserialization	Writes cart objects to files and restores them later.
Multithreading / Synchronization	Runs customer-processing tasks in separate threads and protects shared console input.

1.5 Project Structure

The repository contains the Java source file ShoppingSystem.java. During execution, the program creates customer1.ser, customer2.ser, and customer3.ser files to store the serialized cart objects. These files are generated automatically when the application runs.

Project directory

```
OnlineShoppingCartProject/
|
|-- ShoppingSystem.java
|-- customer1.ser   (generated during execution)
|-- customer2.ser   (generated during execution)
|-- customer3.ser   (generated during execution)
```

Explanation: The .ser files contain serialized OnlineShoppingCart objects for the respective customers.

1.6 Main Components

A. OnlineShoppingCart

This class represents one customer's cart. It stores product names in an ArrayList, provides addProduct() to insert products, and returns an unmodifiable view through getCart(). The class implements Serializable so that the cart object can be written to and restored from a file.

B. OrderProcessingSystem

This class maintains the product-price mapping and calculates the total cart value. The calculateTotal() method uses a stream and mapToDouble() to convert product names into prices and sum them.

C. ProductFilter

ProductFilter is a custom functional interface with one abstract method, genFilter(). It accepts a product name and a numeric value and returns a boolean result. Lambda expressions provide the price and category filtering rules.

D. ShoppingSystem

ShoppingSystem contains the main() method and controls the application flow. It accepts and validates customer input, calculates cart values, applies filters, saves and restores carts, and starts three customer threads. Synchronization ensures that the threads do not read from the shared Scanner at the same time.

2. CODE SNIPPETS AND IMPLEMENTATION

The following snippets are selected from the submitted ShoppingSystem.java source file. They illustrate the principal Java concepts used in the application.

2.1 Imports and Required Packages

```
import java.util.*;
import java.io.*;
import java.util.stream.*;
```

Explanation: java.util supplies collection and input utilities such as List, ArrayList, Map, Collections, and Scanner. java.io supplies file and object-stream classes required for serialization. The Stream API is used to process cart items; it is available through java.util.stream (and is already covered by java.util.* in modern Java, so the explicit stream import is not essential).

2.2 ArrayList-Based Cart

```
class OnlineShoppingCart implements Serializable {

    private static final long serialVersionUID = 1L;
    private final List<String> cart = new ArrayList<>();

    public void addProduct(String product) {
        cart.add(product);
    }

    public List<String> getCart() {
        return Collections.unmodifiableList(cart);
    }
}
```

Explanation: ArrayList stores cart products in insertion order and permits repeated products. The List interface is used as the reference type. Collections.unmodifiableList() prevents callers from modifying the returned view directly.

2.3 Product Price Mapping

```
public static final Map<String, Double> priceMap = Map.of(
    "Laptop", 50000.0,
    "Headset", 25000.0,
    "Iphone", 94000.0
);
```

Explanation: The Map associates each product name with its price, allowing quick lookup by product name. Important implementation note: the submitted code uses Map.of(), which creates an unmodifiable Map; it does not explicitly instantiate HashMap. If the assignment specifically requires HashMap, the implementation should be changed to create a HashMap and populate it with put().

2.4 Calculating the Total Price with Streams

```
public static double calculateTotal(List<String> cart) {
    return cart.stream()
               .mapToDouble(priceMap::get)
               .sum();
}
```

Explanation: stream() processes the cart items as a stream. mapToDouble() looks up each product's price and converts it to a double stream. sum() returns the total value of all products in the cart.

2.5 Functional Interface

```
@FunctionalInterface
interface ProductFilter {
    boolean genFilter(String item, double value);
}
```

Explanation: A functional interface contains exactly one abstract method and can therefore be implemented using a lambda expression. The @FunctionalInterface annotation helps the compiler verify this.

2.6 Lambda Expressions for Filtering

```
ProductFilter onPrice =
    (item, avg) -> OrderProcessingSystem.priceMap.get(item) > avg;

ProductFilter onCategory =
    (item, value) ->
        item.equals("Laptop") || item.equals("Iphone");
```

Explanation: onPrice selects products whose prices exceed the customer's average cart price. onCategory selects Laptop and Iphone as the application's predefined display-device category. The category is encoded through product-name checks rather than stored as a separate product attribute.

2.7 Applying Filters with the Stream API

```
cart.getCart().stream()
    .filter(item -> onPrice.genFilter(item, average))
    .forEach(System.out::println);

cart.getCart().stream()
    .filter(item -> onCategory.genFilter(item, 0))
    .forEach(System.out::println);
```

Explanation: filter() retains only the items that satisfy the supplied functional condition. forEach() prints each matching product.

2.8 Serialization and Deserialization

```
public void saveCart(String filename) {
    try (ObjectOutputStream out =
             new ObjectOutputStream(new FileOutputStream(filename))) {
        out.writeObject(this);
        System.out.println("Cart saved successfully: " + filename);
    } catch (IOException e) {
        System.out.println("Error saving cart: " + e.getMessage());
    }
}

public static OnlineShoppingCart restoreCart(String filename) {
    try (ObjectInputStream in =
             new ObjectInputStream(new FileInputStream(filename))) {
        return (OnlineShoppingCart) in.readObject();
    } catch (IOException | ClassNotFoundException e) {
        System.out.println("Error restoring cart: " + e.getMessage());
        return new OnlineShoppingCart();
    }
}
```

Explanation: Serialization converts the cart object into a byte stream and writes it to a .ser file. Deserialization reads the saved object and reconstructs the cart. Try-with-resources closes the streams automatically.

2.9 Multithreading and Synchronization

```
Object inputLock = new Object();

Thread t1 = new Thread(() -> {
    synchronized (inputLock) {
        processCustomer(1, sc);
    }
});

Thread t2 = new Thread(() -> {
    synchronized (inputLock) {
        processCustomer(2, sc);
    }
});

Thread t3 = new Thread(() -> {
    synchronized (inputLock) {
        processCustomer(3, sc);
    }
});

t1.start();
t2.start();
t3.start();

t1.join();
t2.join();
t3.join();
```

Explanation: Three threads are created to process three customers. The synchronized block ensures that only one thread at a time accesses the shared Scanner input. join() makes the main thread wait until each customer thread finishes.

2.10 List and Map: Suitability Comparison

Operation / Feature	List (ArrayList)	Map
Primary purpose	Stores an ordered sequence of items.	Stores key-value associations.
Access	Access by integer index; searching by value is linear.	Access by key; HashMap typically provides average constant-time lookup.
Duplicates	Allows duplicate product entries.	Keys are unique; a key maps to one value.
Shopping cart use	Suitable for maintaining the sequence of items added to a cart.	Suitable for looking up product information and prices by product name.
Typical operation	Append an item; iterate through cart contents.	Find the price associated with a product name.

In this application, ArrayList is appropriate for the cart because it preserves the order in which items are added and permits the same product to appear multiple times. A Map is appropriate for the product catalogue because a product name can be used as a key to retrieve its price. For a mutable catalogue with the required HashMap implementation, HashMap<String, Double> can be used.

2.11 Program Execution Flow

1. The main method creates a shared Scanner and an input lock.
2. Three customer threads are created and started.
3. Each customer enters the number of products and selects products from the predefined catalogue.

4. The program validates each product name using the product-price Map and adds valid products to the cart.
5. The system calculates the total and average cart prices.
6. Lambda expressions filter products above the average price and products in the display-device category.
7. The cart is serialized to a customer-specific .ser file and then deserialized.
8. The restored cart is displayed, and the main thread waits for all customer threads to finish.

2.12 How to Run the Project

9. Install JDK 11 or later and open a terminal in the project directory.
10. Clone the repository: git clone https://github.com/charanreddy-73/Java-CourseEndProject/
11. Navigate to the source directory containing ShoppingSystem.java.
12. Compile the program: javac ShoppingSystem.java
13. Run the program: java ShoppingSystem
14. Enter the number of products and choose from Laptop, Iphone, and Headset when prompted.

3. RESULTS AND OUTPUT

The application was executed successfully for three customers. The output demonstrates cart creation, price calculation, filtering, invalid-product validation, serialization, deserialization, and completion of all customer-processing threads. The execution order of customer threads may vary.

3.1 Execution Screenshot 1

```
PS C:\Users\reddy>  & 'C:\Users\reddy\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.
1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\reddy\AppD
ata\Local\Temp\vscodesws_ddb8a\jdt_ws\jdt.ls-java-project\bin' 'ShoppingSystem'

Customer 1
Enter number of products: 3
Enter product 1 (Laptop, Iphone, Headset):
Laptop
Enter product 2 (Laptop, Iphone, Headset):
Iphone
Enter product 3 (Laptop, Iphone, Headset):
Headset

Customer 1 Cart:
[Laptop, Iphone, Headset]
Total price: Rs. 169000.0
Average price: Rs. 56333.333333333336

Products above average price:
Iphone

Display devices:
Laptop
Iphone
Cart saved successfully: customer1.ser

Restored Customer 1 Cart:
[Laptop, Iphone, Headset]

Customer 2
Enter number of products: 6
Enter product 1 (Laptop, Iphone, Headset):
Laptop
Enter product 2 (Laptop, Iphone, Headset):
Laptop
Enter product 3 (Laptop, Iphone, Headset):
Laptop
```

Figure 1. Customer 1 cart processing and the beginning of Customer 2 input.

3.2 Execution Screenshot 2

```
Enter product 4 (Laptop, Iphone, Headset):
Laptop
Enter product 5 (Laptop, Iphone, Headset):
Laptop
Enter product 6 (Laptop, Iphone, Headset):
Laptop

Customer 2 Cart:
[Laptop, Laptop, Laptop, Laptop, Laptop, Laptop]
Total price: Rs. 300000.0
Average price: Rs. 50000.0

Products above average price:

Display devices:
Laptop
Laptop
Laptop
Laptop
Laptop
Laptop
Cart saved successfully: customer2.ser

Restored Customer 2 Cart:
[Laptop, Laptop, Laptop, Laptop, Laptop, Laptop]

Customer 3
Enter number of products: 7
Enter product 1 (Laptop, Iphone, Headset):
headset
Invalid product. Try again.
Enter product 1 (Laptop, Iphone, Headset):
Headset
Enter product 2 (Laptop, Iphone, Headset):
Headset
Enter product 3 (Laptop, Iphone, Headset):
Laptop
```

Figure 2. Customer 2 cart processing and the beginning of Customer 3 input.

3.3 Execution Screenshot 3

```
Enter product 4 (Laptop, Iphone, Headset):
Iphone
Enter product 5 (Laptop, Iphone, Headset):
Iphone
Enter product 6 (Laptop, Iphone, Headset):
Headset3
Invalid product. Try again.
Enter product 6 (Laptop, Iphone, Headset):
Headset
Enter product 7 (Laptop, Iphone, Headset):
Iphone

Customer 3 Cart:
[Headset, Headset, Laptop, Iphone, Iphone, Headset, Iphone]
Total price: Rs. 407000.0
Average price: Rs. 58142.857142857145

Products above average price:
Iphone
Iphone
Iphone

Display devices:
Laptop
Iphone
Iphone
Iphone
Cart saved successfully: customer3.ser

Restored Customer 3 Cart:
[Headset, Headset, Laptop, Iphone, Iphone, Headset, Iphone]

All customer orders processed.
```

Figure 3. Customer 3 cart processing, cart restoration, and successful program completion.

3.4 Result Observations

- Customer 1 selected Laptop, Iphone, and Headset. The total was Rs. 169000.0 and the average was approximately Rs. 56333.33. Iphone was above the average price.
- Customer 2 selected six Laptops. The total was Rs. 300000.0 and the average was Rs. 50000.0. No item was strictly above the average.
- Customer 3 entered invalid product text before correcting it. The program rejected the invalid entries and accepted valid product names. The final cart total was Rs. 407000.0.
- The output confirms that each customer's cart was saved to a .ser file and restored with the same product list.
- The final message, 'All customer orders processed.', confirms that the customer threads completed.

4. CONCLUSION

The Online Shopping Cart and Order Processing System demonstrates how core Java features can be combined to implement a basic shopping workflow. ArrayList is used to maintain each customer's cart, while a Map associates product names with their prices. Stream operations simplify price calculation and product filtering, and Lambda Expressions provide concise implementations of the filtering rules through a custom Functional Interface.

Serialization and Deserialization allow customer cart objects to be saved to files and restored later. The use of three customer threads demonstrates concurrent task execution, while synchronization protects access to the shared Scanner input. The collection comparison shows that List and Map serve different purposes: a List is suited to ordered cart contents, whereas a Map is suited to key-based product lookup.

5. PROJECT REPOSITORY

GitHub Repository: https://github.com/charanreddy-73/Java-CourseEndProject

The repository contains the Java project source and project documentation. The .ser files are created when the application is executed.